

Multi – Level linked list

–Insert At End

Program:

```
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}
// --- 2. Insert at End (Main Level) ---
void insertAtEnd()
{
    int data;
    cout << "Enter data: ";
    cin >> data;

    Node *newNode = createNode(data);

    if (head == nullptr)
    {
        head = newNode;
    }
    else
    {
        Node *temp = head;
        while (temp->next != nullptr)
        {
            temp = temp->next;
        }
    }
}
```

```

    }
    temp->next = newNode;
}
cout << "Inserted " << data << " at end." << endl;
}
int main()
{
    int choice;
    while (true)
    {

        cout << "2. Insert at End (Main)" << endl;

        switch (choice)
        {
            case 2:
                insertAtEnd();
                break;
            default:
                cout << "Invalid choice" << endl;
        }
    }
    return 0;
}

```

Program Analysis

Say, int data = 40 .

Now, we pass this data to function:

Node * newNode = createNode(data: 40);

Next,

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64-bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

→ data(an integer) is typically 4 bytes.

→ next(a pointer pointing to another Node) is typically 8 bytes.

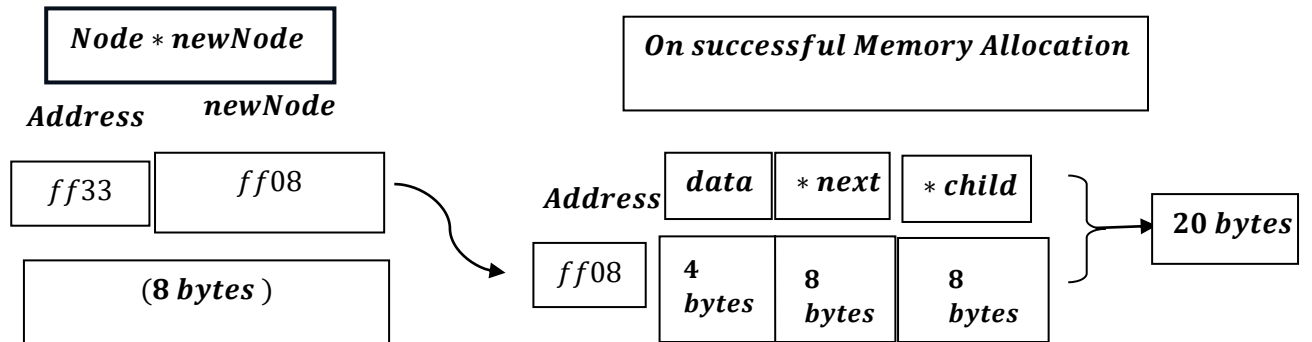
→ child(a pointer pointing to child node) is typically 8 bytes.

→ Therefore, the size of the Node structure is typically 20 bytes (4 bytes for data + 8 bytes for next + 8 bytes for child).

What does it mean?

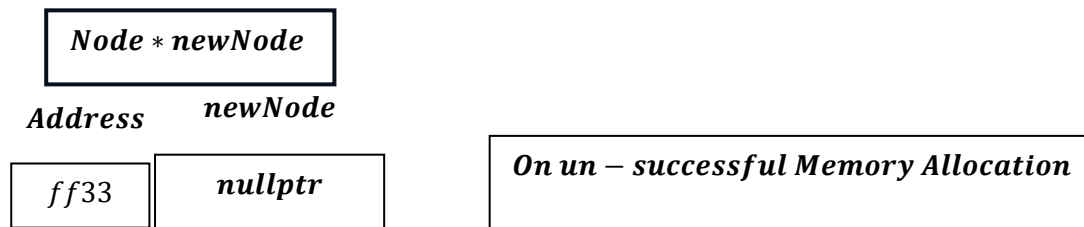
→ newNode will try to allocate memory for integer data for 4 bytes and next pointer variable i.e. typically 8 bytes, also child next pointer variable also typically 8 bytes hence assumed total space consumed is total (4 + 8 + 8) bytes = 20 bytes.

Say,



But on Un – Successful memory allocation:

→ if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer.



Now, if the value of the pointer variable newNode is nullptr, then:

One may output : `Overflow or Memory Allocation Failed as a message.`

Now,

```
int main()
{
    int choice;
    while (true)
    {

        cout << "2. Insert at End (Main)" << endl;

        switch (choice)
        {
            case 2:
                insertAtEnd();
                break;
            default:
                cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

We have give input : 2 as ,through input 2 ,it will help to run insertAtEnd() function , otherwise it will print default case ``Invalid choice``.

```
Node *createNode(int data){...}
```

Return type is struct type pointer variable as mentioned here:

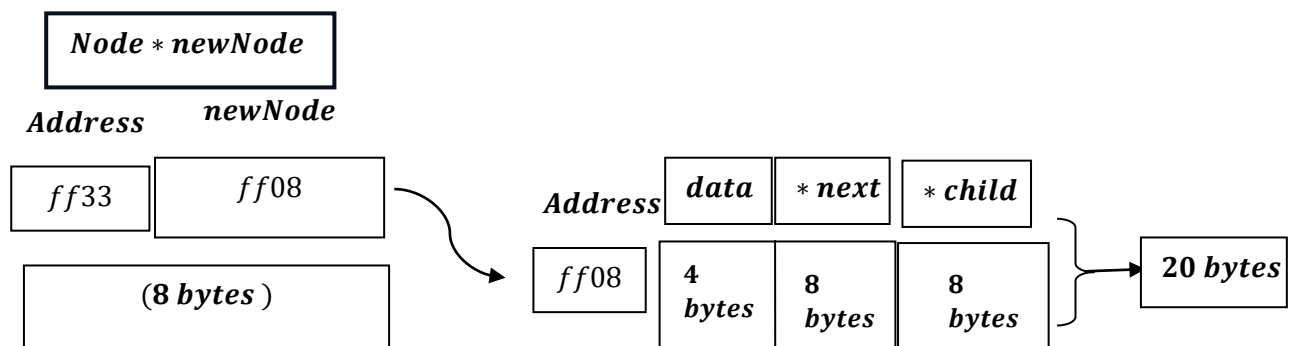
Node * and function name is createNode(variable : 40).

As we run the function inserAtEnd , it takes input data from user i. e. 40 and then , createNode(data: 40).

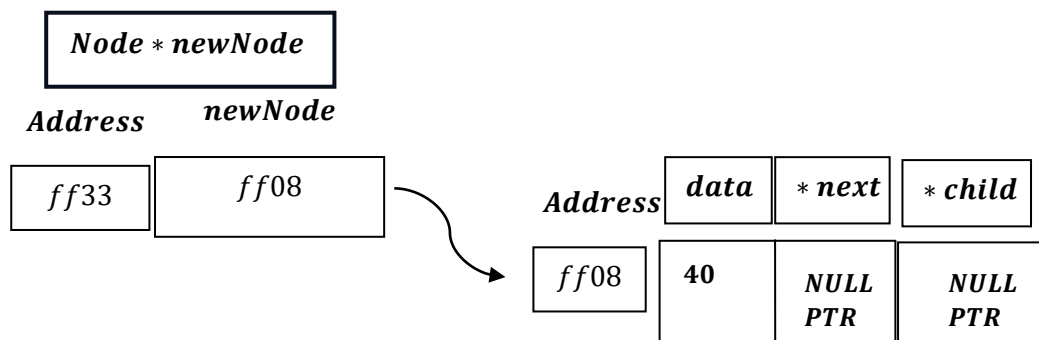
Under function `createNode(data: 40):`

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

We got:



```
newNode->data = data;  
newNode->next = nullptr;  
newNode->child = nullptr;
```



```
return newNode;
```

*As we already know , Return type is struct type pointer variable .
Return newNode will return the value stored inside newNode,
pointer :*

i.e. return newNode : ff08 .

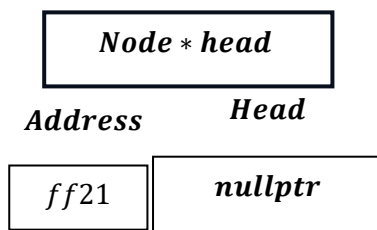
Now under function InsertAtEnd() :

```
Node *newNode = createNode(data);
```

*Node * newNode = ff08.*

Now there are two main condition

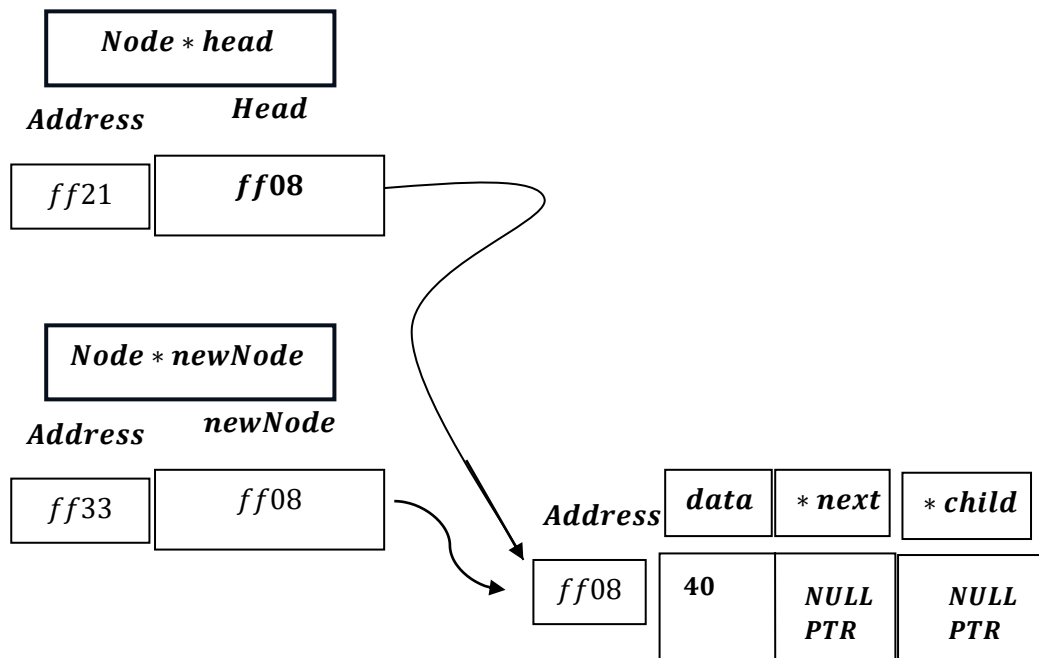
*When Head is assigned to NullPtr [Which means
List is Empty]*



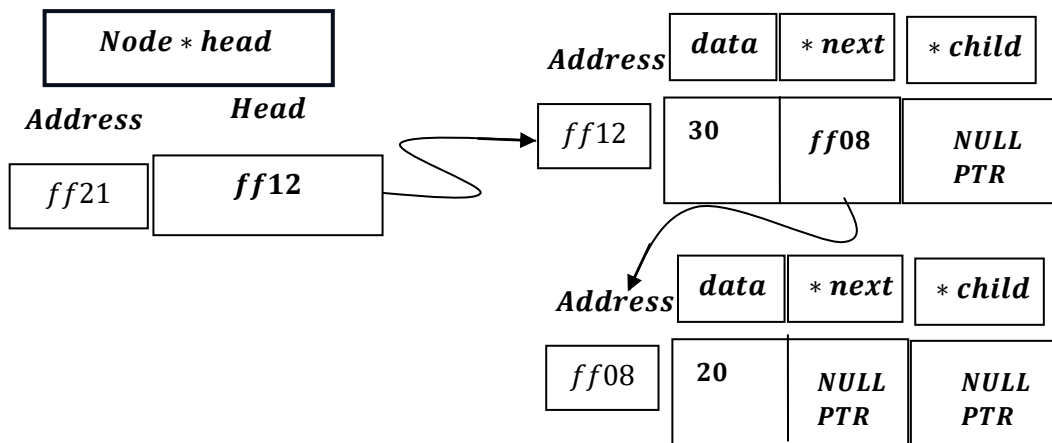
```
if (head == nullptr)
{
    head = newNode;
}
```

That is:

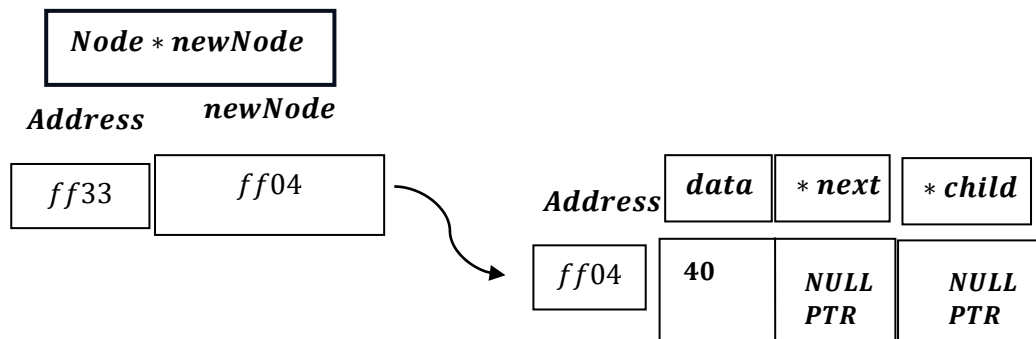
Head = ff08.



2nd condition When List is not Empty Head
is not
NULL



And our newNode is:



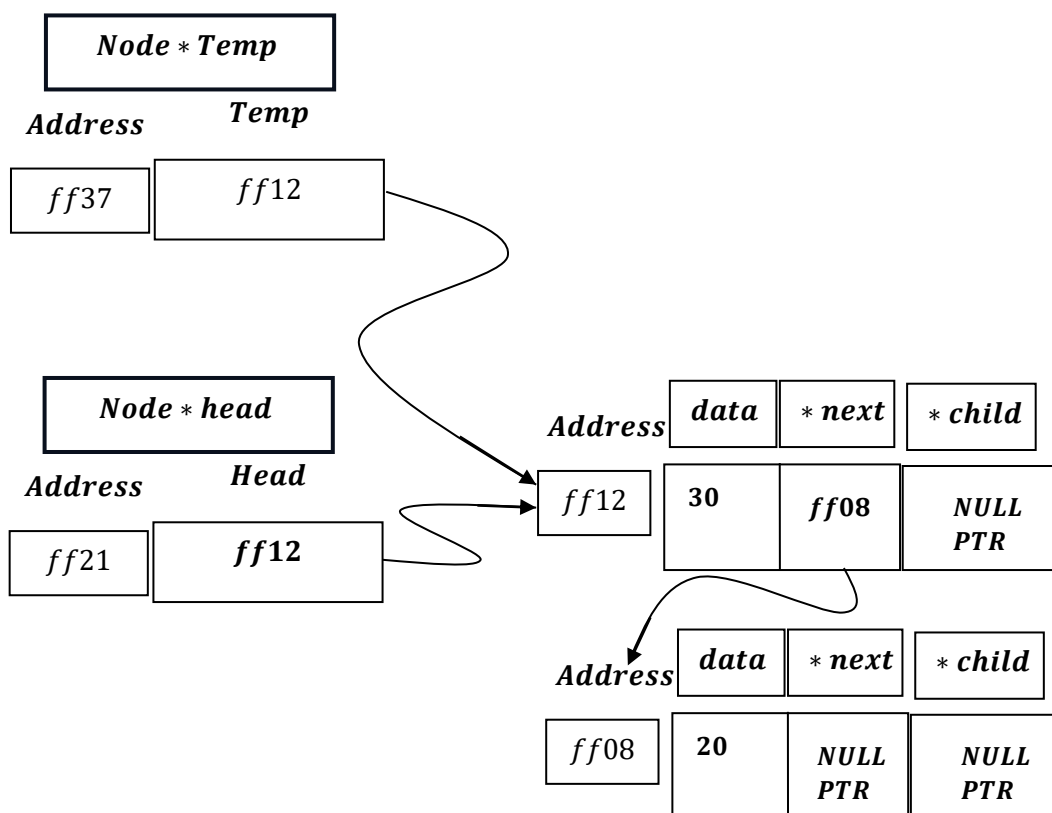
```
else
{
    Node *temp = head;
    while (temp->next != nullptr)
    {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

```
cout << "Inserted " << data << " at end." << endl;
}
```

Now, we need to traverse to the last node.

```
Node *temp = head;
```

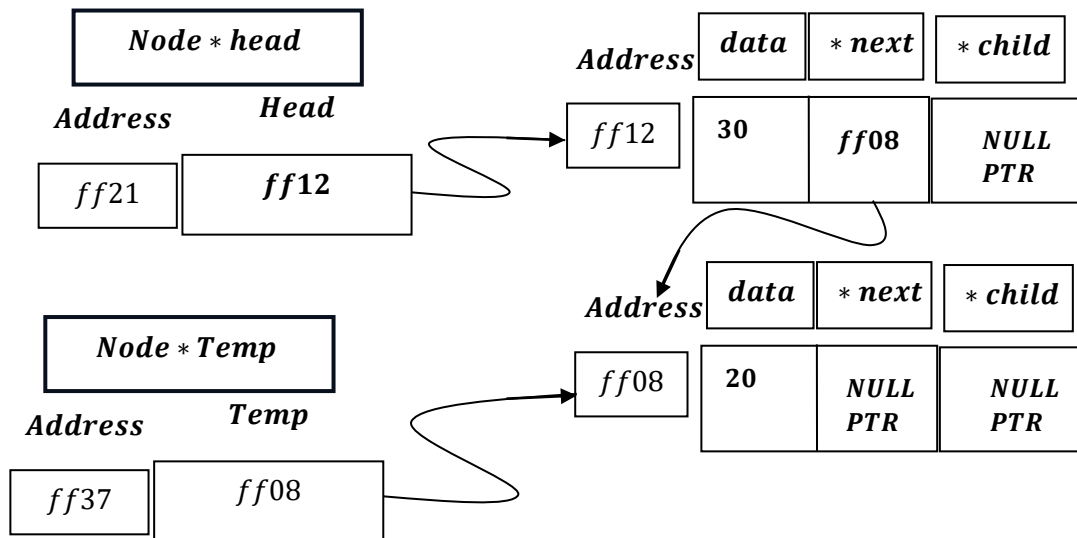
Temp = Head = ff12.



```
while (temp->next != nullptr)
{
    temp = temp->next;
}
```

***Temp* → *Next* = *ff08* ≠ *NULLPTR* (*True*):**

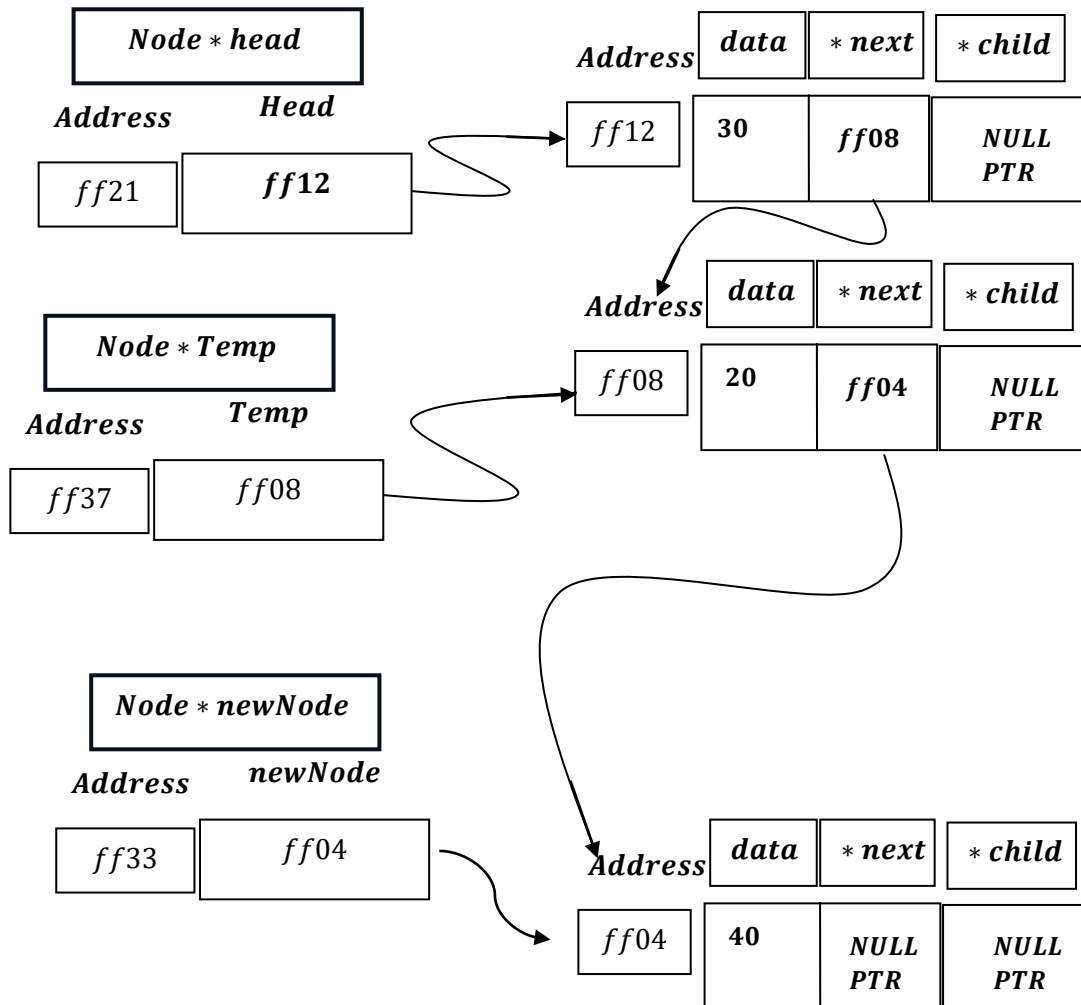
***TEMP* = *TEMP* → *NEXT* = *ff08*.**



***Temp* → *Next* = *NULLPTR* ≠ *NULLPTR* (*FALSE*).**

```
temp->next = newNode;
```

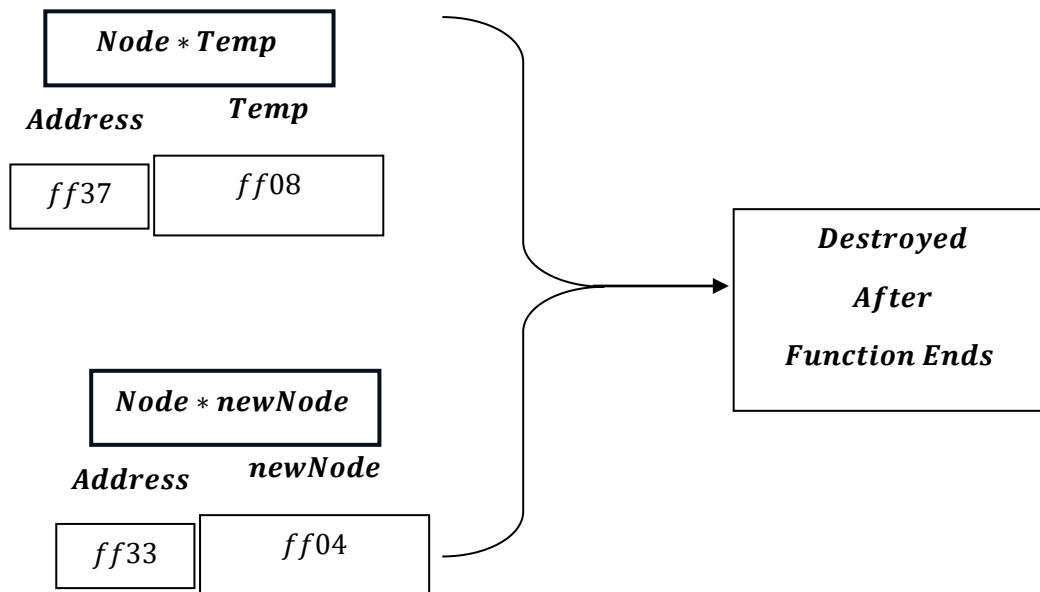
***Temp* → *Next* = *newNode* = *ff04*.**



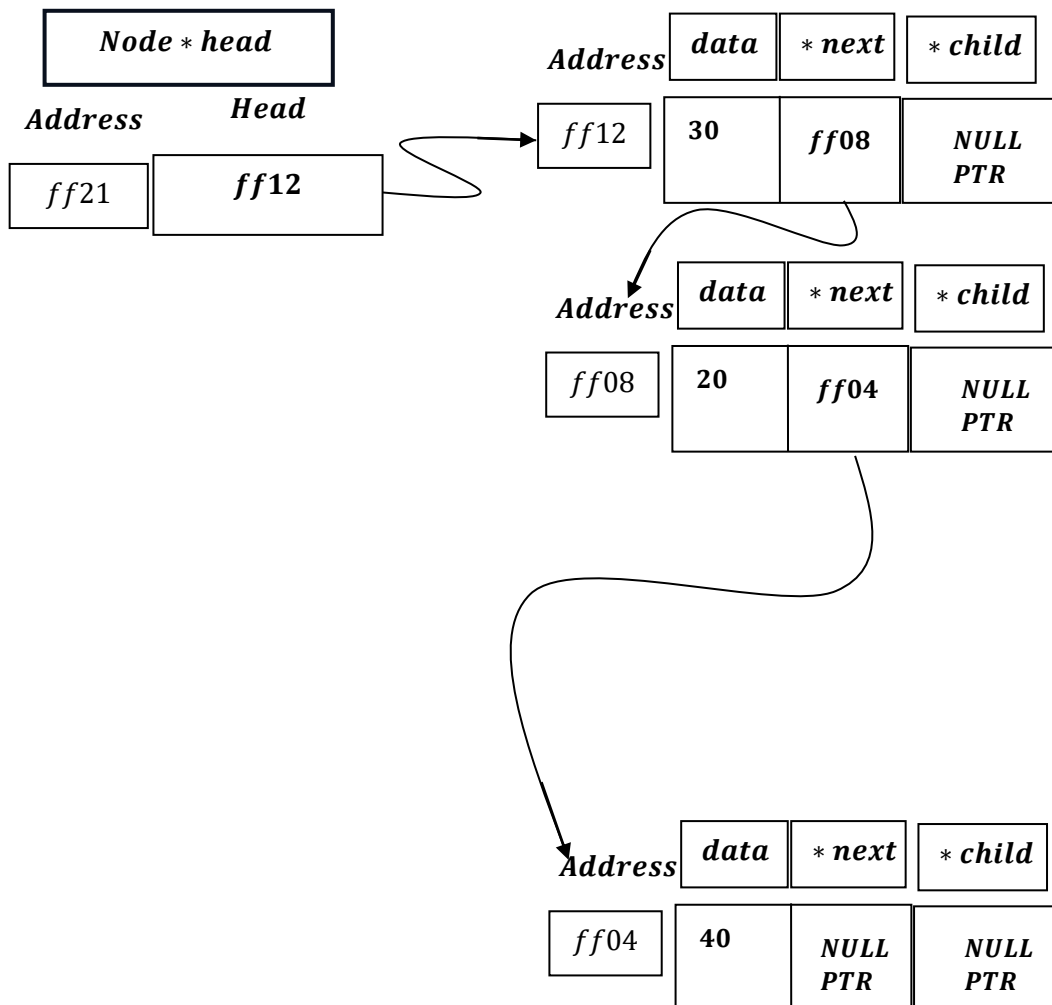
```
cout << "Inserted " << data << " at end." << endl;
```

Print: ``Inserted`` ,40 ,``at the end``.

As the function ends,



And we are left with:



Time Complexity

Best Case : When Head is pointing at NULL i. e.

List is Empty . $\Omega(1)$.

Worst Case: When Head is not pointing at NULL i. e.

List is not Empty. $O(n)$.

Average Case $\approx$ Worst Case (As no true Average Case, Exists). $\Theta(n)$.

<i>Multi – level Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>InsertAt End</i>	<i>1. Best Case</i>	<i>$\Omega(1)$</i>
	<i>2. Worst Case</i>	<i>$O(n)$</i>
	<i>3. Average Case</i>	<i>$\Theta(n)$</i>
